

ABIR DEOL

abir.deol2006@gmail.com · (672) 968-0839 · Edmonton, AB
abirdeol.tech · github.com/apollo-2006 · linkedin.com/in/abirdeol

EDUCATION

University of Alberta · Bachelor of Science, Computing Science *Expected 2028*

TECHNICAL SKILLS

Languages: C, C++17, Python, Go, TypeScript, JavaScript, SQL

Systems: Concurrency and threading, memory management, network programming (TCP, WebSockets), distributed consensus, storage engines

Tools: Linux, Git, Make, GDB, SDL2, FastAPI, React, SQLite, Vite, PyInstaller

PROJECTS

nexus_db · Embedded LSM Key-Value Store | *C++17, Python/FastAPI, React*

- Built an embedded storage engine modeled on LevelDB and RocksDB, using a probabilistic skip-list MemTable for $O(\log n)$ insertion and point lookup without tree-rebalancing overhead.
- Implemented a write-ahead log appending every operation to disk before the MemTable is modified, guaranteeing durability across crashes; frozen MemTables flush to immutable SSTables by sequential write.
- Added tombstone deletion reclaimed during background compaction, and exposed the C++ core to a Python/FastAPI REST layer over a ctypes FFI with a React telemetry dashboard.

nexus_cluster · Distributed Consensus Engine | *Python, Raft, TCP*

- Implemented Raft from scratch (leader election, log replication, and term management over custom RPCs) with no consensus library.
- Managed node lifecycle across Leader, Follower, and Candidate states with election timeouts, keeping a replicated state machine consistent as nodes fail and rejoin.
- Wrote a custom TCP transport layer for inter-node communication, plus a CLI to launch nodes and inspect cluster health.

nexus_editor · Real-Time Collaborative Editor | *Go, React, TypeScript, WebSockets*

- Designed a fractional-indexing CRDT so concurrent edits converge deterministically without centralized locking or an operational-transform server.
- Built a Go WebSocket router spawning an isolated goroutine per client for non-blocking broadcast, with a React 18 and TypeScript frontend reconciling remote payloads in the background.

nano_match · Limit Order Book and Matching Engine | *C++*

- Built a price-time priority matching engine covering the full order lifecycle: submission, queuing, cancellation, and execution.
- Eliminated dynamic heap allocation on the matching hot path using a custom memory pool, targeting high-frequency trading latency constraints.

custom_mem_alloc · Thread-Safe Memory Allocator | *C, pthreads*

- Wrote a malloc replacement requesting one contiguous virtual memory region from the OS via mmap and sub-allocating it with minimal bookkeeping overhead.
- Implemented free-list management with block recycling, 8-byte alignment for correct CPU access, and mutex locking for thread safety.

terminal_dashboard · Real-Time System Monitor | *Python, Rich, SQLite*

- Shipped a terminal system monitor as a standalone Windows executable, reporting per-core CPU, memory, network I/O, and GPU telemetry with rolling history.
- Worked around Windows APIs not exposing AMD GPU sensor data by reading LibreHardwareMonitor's local web server, with automatic WMI fallback; persisted metrics to SQLite.

photon_tracer · CPU Raytracer | *C++*

- Wrote a raytracer from scratch with no graphics libraries: 3D vector math, ray generation, camera positioning, and a modular hittable interface for ray-object intersection.
- Rendered scenes directly to PPM by manipulating raw pixel data, with configurable anti-aliasing and per-pixel sampling to trade render time against fidelity.

Additional: neon_vm (stack-based bytecode VM, C) · rasterizer_engine (software rasterizer, C++17/SDL2) · radiant_slice (tactical FPS with lag compensation, C++) · cpu_rasterizer (browser software rasterizer, JS) · thermal_monitor, valo_scout, cal_cli (Python tooling) · 16 public repositories

ACTIVITIES

Mixed Martial Arts · **Two-Time National Champion** *2020 to 2024*. Trained six days a week alongside full-time coursework.